

Full Length Article

Beyond pairwise dependence: A multi-filter fusion network for graph representation learning

Hui Yan^{a,*}, Ling Guo^a, Guoguo Ai^a, Xin Li^b^a Nanjing University of Science and Technology, Nanjing City, Jiangsu Province, 210094, China^b Nanjing University of Posts and Telecommunications, Nanjing City, Jiangsu Province, 210003, China

ARTICLE INFO

Keywords:

Spectral GNNs
Adaptive frequency
Arbitrary order
Learning filters

ABSTRACT

Polynomial graph filters have been widely used as guiding principles in the design of spectral Graph Neural Networks (GNNs). However, their reliance on pairwise network structures inherently constrains their capabilities. (1) Spectral GNNs tend to suffer from the over-smoothing or over-squashing issues as the polynomial order increases. (2) Overfitting may occur when a spectral filter memorizes a first-order Laplacian matrix of the training graph rather than learning underlying high-order relations. To tackle these issues, we propose a Multi-Filter Fusion Network (MFFN) by performing Fourier expansion on diverse Laplacian matrices of order-incidence. In particular, MFFN encodes eigenvalues with simplicial complexes to yield interactions between different orders in graphs beyond conventional pairwise dependence. Subsequently, MFFN approximates multiple spectral filters with Fourier expansion and semantic-adaptive mask, adapting to various complex graph signals. We conduct extensive experiments to demonstrate that MFFN can learn arbitrary graph spectrum filters and achieve superior performance in both node-level and graph-level classification tasks. Codes and datasets are available at <https://github.com/guoling327/MFFN>.

1. Introduction

Graphs are widely used to represent irregular relationships in various real-world scenarios, such as social networks (Qiu et al., 2018), recommendation systems (Wu et al., 2019), and traffic networks (Li et al., 2018b). To effectively deal with graph-structured data, various graph learning models emerged. Among them, Graph Neural Networks (GNNs) have attracted considerable attention in various graph representation learning tasks, including node classification (Kipf & Welling, 2017), link prediction (Zhang & Chen, 2018), and graph classification (Niepert et al., 2016).

Spectral GNNs are a class of GNN models that implement convolution operations via polynomial spectral filters. For example, ChebyNet (Defferrard et al., 2016) approximates the filters with Chebyshev polynomials. Shortly afterwards, Graph Convolutional Networks (GCN) (Kipf & Welling, 2017) adopts a simplified first-order Chebyshev polynomial, which is proven to be a low-pass filter (Balcilar et al., 2021; Wu et al., 2019; Zhu et al., 2021). Similarly, Approximate Personalized Propagation of Neural Predictions (APPNP) (Gasteiger et al., 2018) utilizes the Personalized PageRank (PPR) mechanism to achieve a low-pass filter. GNN-LF/HF (Zhu et al., 2021) designs two novel deep GNNs to simulate low-frequency and high-frequency filters, respectively. Further-

more, GPR-GNN (Chien et al., 2021) and BernNet (He et al., 2021) employ Monomial bases and Bernstein bases respectively. By learning the coefficients of these bases, they can adapt to different signals and get various spectral filters. In addition, ARMA (Bianchi et al., 2021) learns graph convolution filters using auto-regressive moving average filters.

Existing research has focused on either designing or learning polynomial coefficients to simulate a diverse array of filter types, ranging from simple low-pass and high-pass filters to the more intricate band-rejection and comb filters. Despite their notable achievements, three key challenges remain unresolved. Firstly, high-degree polynomials are often necessary to achieve the required expressive power, as they enable the model to reach the large size of the receptive fields (Chien et al., 2021; Defferrard et al., 2016; He et al., 2021). This enhanced capability is crucial for capturing complex patterns and relationships within the graph structure. Nevertheless, a high degree brings serious performance degradation as shown in Fig. 1. We can see that as the polynomial degree increases, both GPR-GNN and BernNet experience a significant decline in accuracy for node classification. Some researchers attribute it to an over-smoothing issue (Kipf & Welling, 2017; Li et al., 2018a), where node representations become increasingly similar and indistinguishable as more layers are added. Others suggest that the degradation stems from the inability to accurately leverage information encoded in

* Corresponding author.

E-mail addresses: yanhui@njjust.edu.cn (H. Yan), guoling@njjust.edu.cn (L. Guo), guoguo@njjust.edu.cn (G. Ai), xinli@njupt.edu.cn (X. Li).

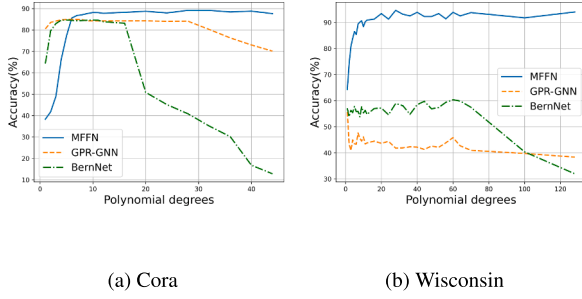


Fig. 1. The performance of spectral GNNs as polynomial filter degrees increases on Cora (homophily graph) and Wisconsin (heterophily graph) in node classification.

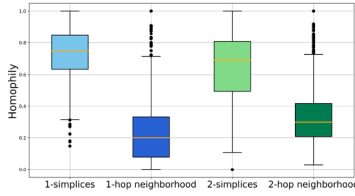


Fig. 2. A comparative analysis of the homophily ratio within different orders of Laplacian and simplicial complex on Chameleon.

high-order connections. This phenomenon is widely recognized as over-squashing (Akansha, 2023).

Next, the existing spectral GNNs are mostly restricted in the homogeneous spectral filtering framework. All input variables, i.e., eigenvalues, of the graph filter are derived from the first-order graph Laplacian (Battiston et al., 2020). Overfitting may occurs when a spectral filter memorizes the topology of the training graph rather than learning diverse regional graph patterns. Additionally, many empirical systems, such as social systems (Centola, 2010), neuronal networks (Ganmor et al., 2011), and ecological networks (Grilli et al., 2017), display group interactions. Such higher-order interactions can hardly be modeled or approximated by a pairwise Laplacian matrix. Heterophilic GNNs (Pei, n.d.) explore high-order adjacency matrices to capture these interactions, but they fail to capture advanced interaction information like cliques and simplices. Recently, several simplicial GNNs have been proposed, replacing the graph Laplacian with the Hodge Laplacian (Schaub et al., 2020) or the Flower-Petals Laplacian (Huang et al., 2024). Two measurements of homophily level with varying hops and simplices on a heterophily graph (Chameleon) are reported in Fig. 2. One is formulated as $NH_i^k = \frac{|\{v_j | v_j \in N_{v_i}^k, y_i = y_j\}|}{|N_{v_i}^k|}$, where $N_{v_i}^k$ denotes the k -hop neighborhood

of v_i . The other one is $NH_i^p = \frac{|\{v_j | v_j \in N_{v_i}^p, y_i = y_j\}|}{|N_{v_i}^p|}$, where $N_{v_i}^p$ represents the p -simplex containing v_i . Homophily represents the tendency of nodes in a network to connect to other nodes that are similar to them, potentially leading to higher classification accuracy (Wang et al., 2024). Observing Fig. 2, it is evident that the median homophily within both 1-simplices and 2-simplices exceeds the median homophily in the 1-hop and 2-hop neighborhoods.

Lastly, the expressive power of pairwise GNNs has been proved to be upper bounded by the WL-test. More recently, various graph Transformers (Dwivedi & Bresson, 2020; Ying et al., 2021) address the limitation of GNNs by encoding position information about nodes. Although position encoding in graph Transformer avoids any structural inductive bias, it ignores structural relationships in complex systems, such as group interactions. Besides, the spatial attention mechanism in the transformer cannot effectively capture the desired frequency response (Bastos et al., 2022).

In order to enhance the flexibility and expressiveness of spectral GNNs, we propose a Multi-Filter Fusion Network for Graph Representa-

tion Learning (MFFN). Specifically, MFFN initially encodes eigenvalues with simplicial complexes to yield interactions between different orders in graphs (Battiston et al., 2020). Such graph spectral rewiring modifies first-order pair-wise connections to mitigate over-squashing. Moreover, simplices of various order act as a graph data augmentation or a regularizer and reduce over-fitting when training data driven models. Subsequently, MFFN approximates multiple spectral filters with Fourier expansion and semantic-adaptive mask. Finally, by combining these bases, MFFN can construct a permutation-equivariant and non-local graph convolution. The main contributions of this paper can be summarized as follows:

- We propose a novel graph spectral filter along with learnable Fourier serials, called MFFN, which can approximate any univariate and multivariate continuous functions. MFFN can easily cope with various constraints on the learned filter via simple reparameterization, such as the positive semidefinite constraint proposed in (He et al., 2021).
- We show that MFFN achieves permutation equivariance, non-locality, and higher approximation ability than current GNNs.
- Extensive experiments on various node-level and graph-level benchmarks demonstrate that MFFN outperforms state-of-the-art GNNs and learns meaningful spectrum patterns.

2. Notations and preliminaries

Notations. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph with node set $\mathcal{V} = \{v_i\}_{i=1}^N$ and edge set $\mathcal{E}$ where the number of nodes is denoted by N . The graph topology is symbolized with an $N \times N$ sized adjacent matrix $\mathbf{A}$ such that $\mathbf{A}(i, j) = 1$ if v_i are v_j linked by an edge, and 0 otherwise. $\mathbf{D}$ denotes a degree matrix of $\mathbf{A}$ with diagonal entries $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$. The normalized Laplacian matrix is defined as $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ where $\mathbf{I}$ is an identity matrix. Using eigendecomposition of $\mathbf{L}$, we have $\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$, where $\mathbf{U}$ is the matrix of eigenvectors and $\mathbf{\Lambda} = \text{diag}[\lambda_1, \lambda_2, \dots, \lambda_N]$ consists of eigenvalues ranged in $[0, 2]$.

2.1. Spectral filters in GNNs

The central idea of spectral GNNs is to transform the graph signal via graph spectral filters. Given an observed N -dimensional graph signal x , we can adopt the following graph regularized problem to recover the clean signal z :

$$\min_z g(z) = (1 - \alpha) z^T \gamma(\mathbf{L}) z + \alpha \|z - x\|_2^2 \quad (1)$$

where $\gamma(\mathbf{L})$ denotes an energy function of $\mathbf{L}$. $\alpha \in [0, 1]$ is a trade-off parameter.

Then $g(z)$ takes the minimum when the derivative $\frac{\partial g(z)}{\partial z} = 0$, which solves to

$$z^* = \mathbf{U} \text{diag} \left[\frac{\alpha}{\alpha + (1 - \alpha)\gamma(\lambda_1)}, \dots, \frac{\alpha}{\alpha + (1 - \alpha)\gamma(\lambda_n)} \right] \mathbf{U}^T x \quad (2)$$

From Eq. (2), we can see z^* is the optimum solution of the objective function in Eq. (1), and $\phi(\lambda) = \frac{\alpha}{\alpha + (1 - \alpha)\gamma(\lambda)}$ is the corresponding optimum filter. Many popular GNNs approximate $\phi(\lambda)$ with a polynomial graph spectral filters $h(\lambda) = \sum_{k=0}^K \theta_k \lambda^k$, i.e., the suffix sum of $\phi(\lambda)$'s Taylor expansion (Defferrard et al., 2016; Kipf & Welling, 2017). For example, ChebNet (Defferrard et al., 2016) learns convolutional filters associated with the Chebyshev basis, expressed as $h(\lambda) = \sum_{k=0}^K \theta_k T_k(\hat{\lambda})$, where $\hat{\lambda} = 2/\lambda_{\max} - 1$ represents the scaled eigenvalue and θ_k denotes the trainable filter coefficients. Vanilla GCN (Kipf & Welling, 2017) simplifies ChebNet by retaining only the first-order term: $h(\lambda) = 1 - \lambda$. GPR-GNN (Defferrard et al., 2016) uses the Monomial basis and obtains $h(\lambda) = \sum_{k=0}^K \theta_k (1 - \lambda)^k$. This can be interpreted as a generalized PageRank. BernNet (He et al., 2021) leverages the Bernstein basis to learn graph spectral filters represented by $h(\lambda) = \sum_{k=0}^K \theta_k \frac{1}{2^k} \binom{K}{k} (1 - \lambda)^{K-k} \lambda^k$.

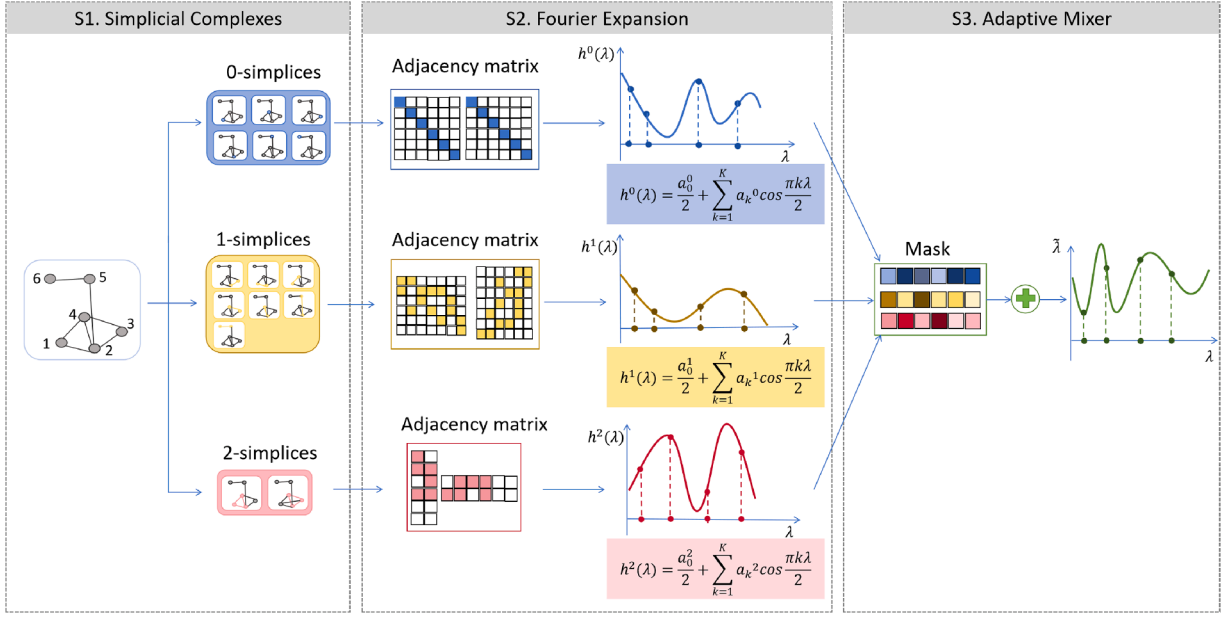


Fig. 3. The framework of our proposed MFFN.

2.2. Positional encoding and eigenvalue encoding

Positional encoding describes the information about the relative or absolute position of the tokens in the sequence (Gehring et al., 2017). Typically, the positional encoding in Transformer (Vaswani et al., 2017) is defined via sine and cosine functions:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (3)$$

where pos is the position of the token in the sequence, i is the dimension index in the positional encoding, and d is the number of dimensions in the positional encoding.

Eigenvalue encoding is similar to positional encoding of Transformer, but they act quite differently. Positional encoding describes the information of discrete positions in the spatial domain, while eigenvalue encoding represents the information of continuous eigenvalues in the spectral domain. Specformer (Bo et al., 2023) use an eigenvalue encoding function as follows:

$$\rho(\lambda, 2i) = \sin\left(\frac{\epsilon \lambda}{10000^{2i/d}}\right)$$

$$\rho(\lambda, 2i+1) = \cos\left(\frac{\epsilon \lambda}{10000^{2i/d}}\right) \quad (4)$$

where i is the dimension of the representations, d is the hidden dimension, and ϵ is a hyperparameter.

3. MFFN: A multi-filter fusion network for graph representation learnings

In this section, we introduce our MFFN model. The overall model architecture is shown in Fig. 3. We first explain how we encode eigenvalues with simplicial complexes to yield rich information embedded in the spectrum. Then we use Fourier expansion as spectral filters to improve the fitting capacity and expressive power. Next, we proceed to pool multi-dimensional eigenvalues using a mask strategy, and subsequently reconstruct the graph Laplacian matrix to enhance graph convolution. Finally, we discuss the relationship between our MFFN and other methods, including MPNNs, polynomial GNNs, and Graph Transformers.

3.1. Eigenvalues encoding with simplicial complexes

To enhance GNNs in their theoretical expressivity, we mine rich information embedding in different orders of simplicial complexes (SCs) (Battiston et al., 2020). A simplicial complex $\mathcal{K}$ on a finite set $\mathcal{M}$ is a collection of subsets of $\mathcal{M}$ that is closed under inclusion. A node subset $w = \{v_0, v_1, \dots, v_p\} \in \mathcal{K}$ with cardinality $p+1$ is called a p -dimensional simplex or simply a p -simplex. We denote the set of all such p -simplices as $\mathcal{K}_p$. Fig. 4 shows an example of generating p -simplex ($p=0,1,2$) on a graph. Geometrically, one can see 0-simplices as vertices, 1-simplices as edges, 2-simplices as triangles, and 3-simplices as tetrahedron. A simplicial complex involves both direct and indirect information. For instance, it directly captures the relationship between v_1 and v_2 by a 1-simplex. Moreover, it naturally captures indirect relationships, such as the relation between v_2 and v_4 by a 1-simplex, and the relation among v_1, v_2 , and v_4 by a 2-simplex, respectively.

The incidence matrix is a classical way to describe the relationships between two classes of objects (Battiston et al., 2020). In a simplicial complex, the incidence matrix describes high-order interactions between nodes and simplices, as shown in Fig. 4. We write the incidence matrices of p -th ($p=0,1,\dots,P$) simplices as $\mathbf{H}_p \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{K}_p|}$ to describe the association between nodes and p -simplices, with entry $\mathbf{H}_p(v, w) = 1$ indicating the node v in $\mathcal{G}$ is contained in the node subset w of the p -simplex. For example, $\mathbf{H}_0$ is the node-to-node incidence matrix, $\mathbf{H}_1$ is the node-to-edge incidence matrix, and $\mathbf{H}_2$ is the node-to-triangles incidence matrix.

Given an incidence matrices $\mathbf{H}_p$, the p -th order adjacency matrices of inter-order simplices is defined as:

$$\mathbf{A}_p = \mathbf{D}_p^{-1/2} \mathbf{H}_p \mathbf{H}_p^T \mathbf{D}_p^{-1/2} \quad (5)$$

where $\mathbf{D}_p$ is the diagonal degree matrix of $\mathbf{H}_p \mathbf{H}_p^T$.

The normalized p -th order Laplacian matrix based on the constructed $\mathbf{A}_p$ is defined as:

$$\mathbf{L}_p = \mathbf{I} - \mathbf{D}_p^{-1/2} \mathbf{A}_p \mathbf{D}_p^{-1/2} \quad (6)$$

As a symmetric Laplacian matrix, $\mathbf{L}_p$ can further be decomposed as

$$\mathbf{L}_p = \mathbf{U}_p \mathbf{\Lambda}_p \mathbf{U}_p^T \quad (7)$$

where $\mathbf{\Lambda}_p = \text{diag}[\lambda_1^p, \lambda_2^p, \dots, \lambda_N^p]$.

Now, we encode each eigenvalue as $g(\lambda_i) = [\lambda_i^1, \lambda_i^2, \dots, \lambda_i^P]$ ($i = 1, 2, \dots, N$) with the mapping function $g(\lambda_i) : \mathbb{R}^1 \rightarrow \mathbb{R}^P$.

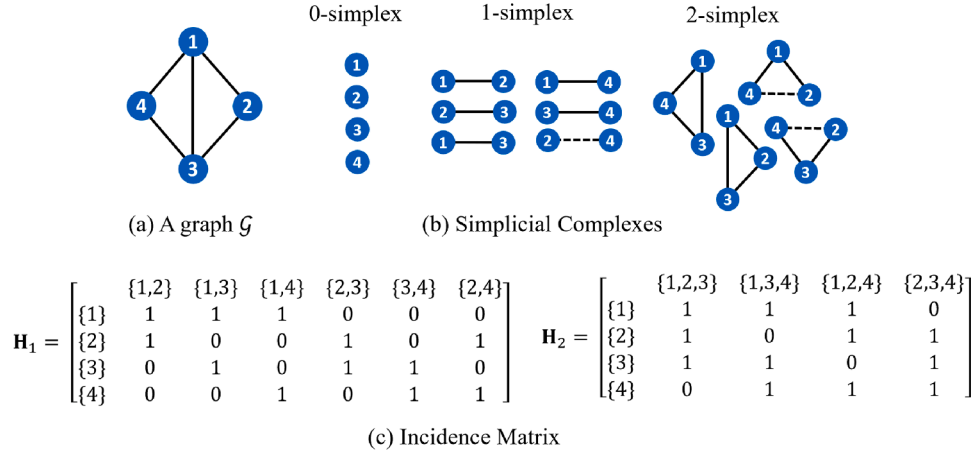


Fig. 4. An example of generating simplicial complex and incidence matrix from a graph. (a) A toy graph $\mathcal{G}$. (b) 0-, 1- and 2-simplex generated from $\mathcal{G}$. (c) $\mathbf{H}_1$ and $\mathbf{H}_2$ are the node-to-edge and node-to-triangle incidence matrix generated from $\mathcal{G}$, respectively.

3.2. Fourier expansion of spectral filters

A Fourier series is a Fourier expansion of a periodic function $f(x)$ in terms of an infinite sum of sines and cosines (Zill, 2020). The formula for Fourier series of a function $f(x)$ in the interval $[-l, l]$ is given by:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} a_k \cos\left(\frac{\pi k x}{l}\right) + \sum_{k=1}^{\infty} b_k \sin\left(\frac{\pi k x}{l}\right) \quad (8)$$

here, a_0 , a_k and b_k are Fourier coefficients. These coefficients are calculated through the following formulas:

$$\begin{cases} a_0 = \frac{1}{2l} \int_{-l}^l f(x) dx \\ a_k = \frac{1}{2l} \int_{-l}^l f(x) \cos\left(\frac{\pi k x}{l}\right) dx \\ b_k = \frac{1}{2l} \int_{-l}^l f(x) \sin\left(\frac{\pi k x}{l}\right) dx \end{cases}$$

In the field of approximation theory, it is established that the Fourier series achieves near-optimum error when approximating a periodic function. To enhance the fitting capability, we replace Taylor expansion of the exact spectral filter $\phi(\lambda)$ in Eq. (2) with Fourier expansion. But an arbitrary spectral filter $\phi(\lambda)$ in Eq. (2) or its approximation $h(\lambda)$ is not guaranteed to be periodic. To solve this problem, we design $H(\lambda)$ to be an even periodic extension of $h(\lambda)$ as:

$$H(\lambda) = \begin{cases} h(\lambda) & 0 \leq \lambda \leq 2 \\ h(-\lambda) & -2 \leq \lambda < 0 \end{cases} \quad (9)$$

After periodic extension, we get the Fourier series of $h(\lambda)$ by retaining that of $H(\lambda)$ in the interval $[0, 2]$ as below:

$$h(\lambda) = H(\lambda) = \frac{a_0}{2} + \sum_{k=1}^{\infty} a_k \cos\left(\frac{\pi k \lambda}{2}\right) \quad (10)$$

Note that there are only cosines bases in the function, as $H(\lambda)$ is an even function with the y -axis as the axis of symmetry.

To filter out the distinctive characteristics from different simplicial complexes, we adopt Fourier series with discrete Fourier coefficients a_k^p for the p -th simplicial complex ($p = 0, 1, \dots, P$):

$$h^p(\lambda) = \frac{a_0^p}{2} + \sum_{k=1}^K a_k^p \cos\left(\frac{\pi k \lambda}{2}\right) \quad (11)$$

Replacing $h^p(\lambda)$ in Eq. (11) with $h^p(\lambda_i^p)|_{i=1}^N$, we have

$$\begin{aligned} h^p(\Lambda_p) &= \text{diag}[h^p(\lambda_1^p), \dots, h^p(\lambda_N^p)] \\ &= \text{diag}\left[\frac{a_0^p}{2} + \sum_{k=1}^K a_k^p \cos\left(\frac{\pi k \lambda_1^p}{2}\right), \dots, \frac{a_0^p}{2} + \sum_{k=1}^K a_k^p \cos\left(\frac{\pi k \lambda_N^p}{2}\right)\right] \end{aligned} \quad (12)$$

where coefficients $a_i^p|_{i=0}^K$ can be learned from the graph structure and signal in an end-to-end fashion.

3.3. Eigenvalues decoding with adaptive mixer

After getting P filtered diagonal matrix of eigenvalues, we reconstruct individual graph-Laplacian matrices, and then filter graph signal x as follows:

$$z_p = \mathbf{U}_p h^p(\Lambda_p) \mathbf{U}_p^T x \quad (13)$$

For mixing representation from multi-order Laplacian matrix, we concatenate them along the order, then we add a fully connected layer followed by a softmax layer.

$$\mathbf{Z} = \left\| \begin{matrix} z_p \\ \vdots \\ z_P \end{matrix} \right\|_{p=0}^P \quad (14)$$

$$\mathbf{M} = \text{SoftMax}(\mathbf{Z}\mathbf{W})$$

where $\mathbf{W} \in \mathbb{R}^{(P+1) \times (P+1)}$ are a correlation matrix to be learnt. $\mathbf{M}$ is the node-adaptive mask matrix learned from all simplicial complexes, which has the shape of $\mathbb{R}^{N \times (P+1)}$.

Finally, the convolutional operator of our MFFN is defined as follows:

$$z = \frac{1}{P+1} \sum_{p=0}^P \mathbf{U}_p (\text{diag}(\mathbf{M}_{:,p}) \odot \Lambda^p) \mathbf{U}_p^T x \quad (15)$$

where $\mathbf{M}_{:,p}$ denotes the p -th column of $\mathbf{M}$. $\odot$ denotes Hadamard product, also known as elementwise multiplication.

Alternatively, to reduce the computation complexity of matrix multiplication, we use the following node-wise attention:

$$z = \frac{1}{P+1} \sum_{p=0}^P \text{diag}(\mathbf{M}_{:,p}) z_p \quad (16)$$

3.4. Analysis

Compared to Taylor expansion in most spectral GNNs, Fourier expansion in our MFFN has the following benefits when being used to define graph convolution:

(1) **Orthogonal basis.** The orthogonal polynomial bases could learn graph filters more effectively (Wang & Zhang, 2022). Chebyshev basis is only orthogonal w.r.t. a specific weight function. Monomial basis cannot be orthogonal to any weight function. Bernstein is not even orthogonal. In contrast, the sine and cosines bases in Fourier expansion are orthogonality (Foupouagnigni, 2020), guaranteeing superior convergence rate on filter learning (Wang et al., 2018) in the field of approximation theory.

(2) **Convergence guarantee.** Taylor's series is conditional on the function being derivable of any order, while Fourier's series only requires that the function not have a Type II discontinuity. The latter condition is significantly more relaxed. Besides, if we substitute $\hat{\lambda} \in [-1, 1]$

Table 1Node regression results, mean of the sum of squared error & R^2 score on synthetic data.

	Low-pass $\exp(-10\lambda^2)$	High-pass $1-\exp(-10\lambda^2)$	Band-pass $\exp(-10(\lambda-1)^2)$	Band-rejection $1-\exp(-10(\lambda-1)^2)$	Comb $ \sin(\pi\lambda) $
GCN	3.4799(0.9872)	67.6635(0.2364)	25.8755(0.1148)	21.0747(0.9438)	50.5120(0.2977)
GAT	2.3574(0.9905)	21.9618(0.7529)	14.4326(0.4823)	12.6384(0.9652)	23.1813(0.6957)
GPR-GNN	0.4169(0.9984)	0.0943 (0.9986)	3.5121 (0.8551)	3.7917 (0.9905)	4.6549 (0.9311)
ARMA	1.8478(0.9932)	1.8632 (0.9793)	7.6922 (0.7098)	8.2732 (0.9782)	15.1214(0.7975)
ChebNet	0.8220(0.9973)	0.7867 (0.9903)	2.2722 (0.9104)	2.5296 (0.9934)	4.0735 (0.9447)
BernNet	0.0314(0.9999)	0.0113 (0.9999)	0.0411 (0.9984)	0.9313 (0.9973)	0.9982 (0.9868)
Specformer	0.0004(0.9999)	0.0026 (0.9999)	0.0017 (0.9999)	0.0024 (0.9999)	0.0057 (0.9999)
MFFN	0.0004(1.0000)	0.0024 (1.0000)	0.0008 (1.0000)	0.0024 (1.0000)	0.0048 (1.0000)

for $\cos(\hat{\lambda}) \in [-1, 1]$, Chebyshev polynomials becomes a polynomial of $\cos(\lambda)$. Thus, Chebyshev basis is a special case of MFFN. In the field of approximation theory, it is established that the Fourier series achieves the optimum convergent rate for approximating a function.

(3) **Positive semidefinite.** As it turns out, $\gamma(\mathbf{L})$ in Eq. (1) has to be positive semidefinite. Otherwise, $g(z)$ is not convex, and the solution to $\frac{\partial g(z)}{\partial z} = 0$ may correspond to a saddle point. By the positive semidefinite constraint, we have to guarantee $h^p(\lambda) \in [0, 1]$ when $\lambda \in [0, 2]$ in Eq. (9). It is trivial, as we can always rescale each $a_i|_{i=1}^K$ by a factor of $\frac{1}{2K}$ and then add a constant of $\frac{1}{2}$ to $h^p(\lambda)$ since $\cos(\cdot)$ is between -1 and 1.

(4) **Global convolution.** MFFN captures the global information due to fitting all N eigenvectors. Besides, escape from multiple orders of the adjacency matrix in polynomial function, our MFFN can break the over-smoothing of polynomial GNNs.

Complexity. Our model has two parts of computation: spectral decomposition and forward process. (1) Spectral decomposition is pre-computed and has the complexity of $O(N^3)$ where N represents the number of nodes. The spectral decomposition only needs to be calculated once and can be reused in the forward process, reducing the overall complexity. (2) The forward complexity has four parts: Fourier expansion, learnable bases, graph convolution, and mask. Their corresponding complexities are $O(K)$, $O(PN^2)$, $O(Nd)$ and $O(Nd)$, respectively, where P , K , d denotes simplicial complexes, Fourier series terms, and the feature dimension for each node, respectively. The overall forward complexity is $O(PN^2 + Nd + K)$. To reduce computational complexity, we can use the Sparse Generalized Eigenvalue (SGE) algorithms (Cai et al., 2021) and use truncated spectral decomposition, where $Q(Q \ll N)$ eigenvalues and eigenvectors need to be calculated, in which case the forward complexity will reduce to $O(PQ^2 + Nd + K)$.

4. Experiment

We conduct experiments on learning filters, node classification tasks, and graph classification tasks to verify the effectiveness of MFFN. All experiments are performed by running PyTorch on TITAN Xp GPU machines equipped with 12GB memory. Codes and data are available at <https://github.com/guoling327/MFFN>.

4.1. Learning filters on synthetic data

Dataset. Following Bo et al. (2023), He et al. (2021), we perform an empirical analysis on 50 real images with a resolution of 100×100 sourced from the Image Processing Toolbox in Matlab. Each image is treated as a 2D regular grid graph with 4-neighborhood connectivity, consistent with the experimental setup described in Balcilar et al. (2021). The graph structure is represented by a $10,000 \times 10,000$ adjacency matrix, while the pixel intensities are represented by a 10,000-dimensional signal vector. We apply five distinct filters (low-pass, high-pass, band-pass, band-rejection, and comb) to the spectral domain of the signal for each image.

Experimental Setting. We utilize the initial graph signal as input and the filtering signal for training supervision. By learning the correct filter, we aim to minimize the square error between the output and the

filtering signal. We evaluate MFFN against six GNNs as baselines: GCN (Kipf & Welling, 2017), GAT (Veličković et al., 2018), ChebyNet (Defferrard et al., 2016), GPR-GNN (Chien et al., 2021), BernNet (He et al., 2021), and Specformer (Bo et al., 2023). We adjust the number of hidden units to maintain nearly 2000 trainable parameters for each method and set the polynomial order to 10 for ChebyNet, GPR-GNN, BernNet, and our model.

During the training process, we set the maximum number of epochs to 2000, implementing early stopping if there is no reduction in loss over 200 consecutive epochs. All regularization techniques have been omitted, and a constant learning rate of 0.01 is used across all models. We assess each method using two evaluation metrics: the sum of squared errors (where lower values are preferred) and the R^2 score (where higher values are favored).

Results. The experiment results are shown in Table 1, which indicates that MFFN performs best on all synthetic graphs. Especially, it has more improvements on challenging graphs, such as band-rejection and comb. This validates the effectiveness of MFFN in learning complex graph filters. In addition, we find that GCN and GAT are limited to handling low-pass filters. This limitation may be due to the inherent ease of fitting low-pass filters. Meanwhile, GPR-GNN, ARMA, ChebNet, and BernNet are capable of learning various filters from the signals, with Specformer showing superior performance on complex signal types. However, their overall expressiveness remains inferior to that of MFFN.

We visualize the graph filters learned by GPR-GNN, BernNet, Specformer, and MFFN in Fig. 5, where the horizontal axis presents the original eigenvalues, and the vertical axis indicates the corresponding new eigenvalues. It can be seen that all methods can fit the easy filters well, i.e., Low-pass and High-pass. However, the polynomial- and Transformer-based GNNs cannot learn the valleys in Band-pass, Band-rejection, and Comb. In addition, in Band-pass and Band-rejection, GPR-GNN encounters a bad Runge phenomenon, characterized by pronounced oscillations near the interval's endpoints. On the contrary, MFFN fits the ground truth precisely, reflecting the superior learning ability over polynomials.

4.2. Node classification

Datasets. We use 11 real-world graph datasets to evaluate our model MFFN on the semi-supervised node classification task. The datasets include four homophily graphs: Cora, Citeseer, Photo, and Computers, as well as seven heterophily datasets including Cornell, Chameleon, Wisconsin, Texas, Squirrel, Actor, and Penn94. The statistics are summarized in Table 2.

Experimental Setting. We use the public data segmentation provided by Pei (n.d.), averaging an average train/val/test split ratio of 60%/20%/20% nodes per class. To ensure fairness, all models report means with standard deviations calculated over 10 random splits. We use the Adam optimizer (Diederik, 2015) and 1000 epochs to train our model and baselines. For baselines, we use their default hyperparameter settings as they generally lead to optimal results. For our proposed method, we fix the hidden cell to 64. The fol-

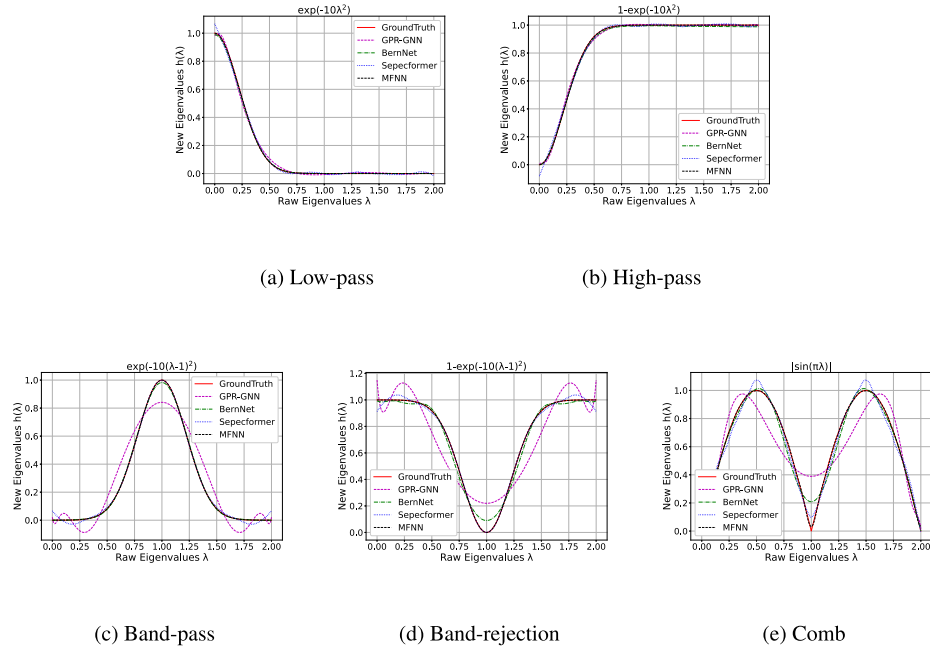


Fig. 5. Illustrations of five complex filters and their approximations learned by our MFFN and other spectral GNNs.

Table 2

The statistics of node classification datasets.

	Cora	Citeseer	Photo	Computers	Cornell	Chameleon	Wisconsin	Texas	Squirrel	Actor	Penn94
Graphs	1	1	1	1	1	1	1	1	1	1	1
Nodes	2708	3327	7,650	13,725	183	2,277	251	183	5,201	7600	41,554
Edges	5,278	4552	119,081	245,861	277	31,371	515	279	198,353	26,659	1,362,229
Features	1433	3703	745	767	1703	2325	1703	1703	2089	932	4,814
Classes	7	6	8	10	5	5	5	5	5	5	2

Table 3

Node classification accuracy(%) $\pm$ 95% confidence interval of MFFN on different datasets compared with baselines.

	Homophily				Heterophily						
	Cora	Citeseer	Photo	Computers	Cornell	Chameleon	Wisconsin	Texas	Squirrel	Actor	Penn94
MLP	77.01 $\pm$ 1.05	76.51 $\pm$ 0.70	89.66 $\pm$ 0.44	80.66 $\pm$ 0.48	90.98 $\pm$ 2.62	47.57 $\pm$ 1.31	93.88 $\pm$ 2.12	91.64 $\pm$ 2.13	31.09 $\pm$ 0.79	39.08 $\pm$ 0.70	73.61 $\pm$ 0.38
GCN	88.13 $\pm$ 0.82	79.82 $\pm$ 0.57	93.46 $\pm$ 0.40	87.04 $\pm$ 0.36	67.38 $\pm$ 4.59	58.16 $\pm$ 1.66	66.25 $\pm$ 2.75	76.89 $\pm$ 3.44	45.65 $\pm$ 0.85	32.10 $\pm$ 1.23	81.24 $\pm$ 0.45
GAT	88.23 $\pm$ 0.74	80.56 $\pm$ 0.59	94.26 $\pm$ 0.35	89.03 $\pm$ 0.60	79.51 $\pm$ 2.13	61.79 $\pm$ 0.92	71.88 $\pm$ 2.62	80.66 $\pm$ 1.80	44.27 $\pm$ 0.86	32.12 $\pm$ 2.13	81.32 $\pm$ 0.63
GCN-JKNet	87.37 $\pm$ 1.31	77.79 $\pm$ 0.59	91.97 $\pm$ 0.87	70.67 $\pm$ 6.77	67.54 $\pm$ 3.12	38.25 $\pm$ 1.27	64.13 $\pm$ 4.63	75.41 $\pm$ 4.43	53.08 $\pm$ 0.76	31.61 $\pm$ 1.57	81.05 $\pm$ 0.94
Mixhop	83.10 $\pm$ 0.90	72.70 $\pm$ 1.33	90.76 $\pm$ 1.12	86.46 $\pm$ 0.85	63.44 $\pm$ 6.39	54.97 $\pm$ 1.58	77.62 $\pm$ 4.00	74.26 $\pm$ 3.44	33.44 $\pm$ 1.97	34.68 $\pm$ 2.84	82.63 $\pm$ 0.83
H2GCN	86.98 $\pm$ 0.74	79.97 $\pm$ 0.61	93.56 $\pm$ 0.25	88.83 $\pm$ 0.42	88.69 $\pm$ 1.48	58.11 $\pm$ 1.03	91.50 $\pm$ 1.88	87.05 $\pm$ 2.63	38.31 $\pm$ 0.93	33.79 $\pm$ 0.51	80.65 $\pm$ 0.75
APNP	87.82 $\pm$ 0.85	80.78 $\pm$ 0.65	92.89 $\pm$ 0.31	84.18 $\pm$ 0.49	89.67 $\pm$ 2.13	51.57 $\pm$ 1.14	93.88 $\pm$ 2.12	91.64 $\pm$ 1.81	34.09 $\pm$ 0.80	39.05 $\pm$ 0.68	78.26 $\pm$ 0.15
GPR-GNN	88.72 $\pm$ 0.97	79.85 $\pm$ 0.86	93.68 $\pm$ 0.37	88.95 $\pm$ 0.59	91.60 $\pm$ 1.64	66.77 $\pm$ 1.62	92.50 $\pm$ 2.13	92.95 $\pm$ 1.31	50.15 $\pm$ 0.78	39.27 $\pm$ 0.67	82.56 $\pm$ 0.19
BernNet	88.55 $\pm$ 1.10	80.38 $\pm$ 0.86	93.56 $\pm$ 0.38	87.57 $\pm$ 0.38	89.67 $\pm$ 1.80	67.11 $\pm$ 1.81	94.00 $\pm$ 1.87	93.12 $\pm$ 0.65	52.81 $\pm$ 1.22	39.89 $\pm$ 0.65	82.47 $\pm$ 0.21
HiGCN	88.63 $\pm$ 0.28	80.47 $\pm$ 0.31	94.10 $\pm$ 0.24	90.35 $\pm$ 0.31	89.90 $\pm$ 1.80	68.47 $\pm$ 0.45	94.12 $\pm$ 0.68	92.15 $\pm$ 0.73	51.86 $\pm$ 0.42	41.03 $\pm$ 0.24	81.52 $\pm$ 0.84
Transformer	71.83 $\pm$ 1.68	70.55 $\pm$ 1.20	90.05 $\pm$ 1.50	88.78 $\pm$ 0.50	84.75 $\pm$ 2.62	46.39 $\pm$ 1.97	80.75 $\pm$ 7.75	84.75 $\pm$ 2.30	31.90 $\pm$ 3.16	39.95 $\pm$ 1.64	OOM
Graphormer	72.71 $\pm$ 0.78	73.30 $\pm$ 1.21	91.20 $\pm$ 4.12	OOM	82.01 $\pm$ 5.12	54.49 $\pm$ 3.11	81.75 $\pm$ 2.75	86.88 $\pm$ 0.98	36.96 $\pm$ 1.75	38.45 $\pm$ 1.38	OOM
NAGphormer	79.97 $\pm$ 1.11	75.30 $\pm$ 0.80	93.33 $\pm$ 0.11	88.46 $\pm$ 1.11	78.69 $\pm$ 0.90	54.92 $\pm$ 1.11	81.25 $\pm$ 0.63	91.80 $\pm$ 1.11	48.55 $\pm$ 2.56	40.08 $\pm$ 1.50	73.98 $\pm$ 0.53
MFFN	89.18$\pm$0.89	80.90$\pm$0.76	95.18$\pm$0.33	91.05$\pm$0.26	91.64$\pm$2.62	70.72$\pm$1.18	94.63$\pm$1.50	93.61$\pm$2.13	59.91$\pm$0.85	41.07$\pm$1.71	82.64$\pm$0.51

lowing hyperparameters are adjusted for our proposed model: learning rate $\in \{0.1, 0.2, 0.5, 0.01, 0.02, \dots, 0.09, 0.001, \dots, 0.005\}$, dropout rate $\in \{0.0, 0.5, 0.6, 0.7\}$, and weight decay $\in \{0.0, 5e^{-3}, 5e^{-4}, 5e^{-5}, 5e^{-6}\}$. For the large-scale dataset Penn94, we implement truncated spectral decomposition to enhance scalability, using eigenvectors corresponding to the smallest 3000 (low-frequency) and the largest 3000 (high-frequency) eigenvalues. We evaluate the performance of all models using accuracy (Liu & Wu, 2010) as a measure, which represents the percentage of nodes where the labels are correctly classified.

Baselines. We compare MFFN with the following benchmarks: (1) MLP; (2) GCN and its variables: GCN (Kipf & Welling, 2017), GAT

(Veličković et al., 2018), JKNet (Xu et al., 2018), MixHop (Abu-El-Hajja et al., 2019), H2GCN (Zhu et al., 2020); (3) spectral-based GNNs: APPNP (Gasteiger et al., 2018), GPR-GNN (Chien et al., 2021), BernNet (He et al., 2021) and HiGCN (Huang et al., 2024); (4) graph Transformers: Transformer (Vaswani et al., 2017), Graphormer (Ying et al., 2021), NAGphormer (Chen et al., 2023).

Results. We show in Table 3 the average accuracy and variance of the MFFN compared with the other benchmark models in the real-world datasets. “OOM” means a lack of memory and bold indicates the best results. Our method outperforms all baselines on ten datasets, and the performance demonstrates that our model effectively captures graph in-

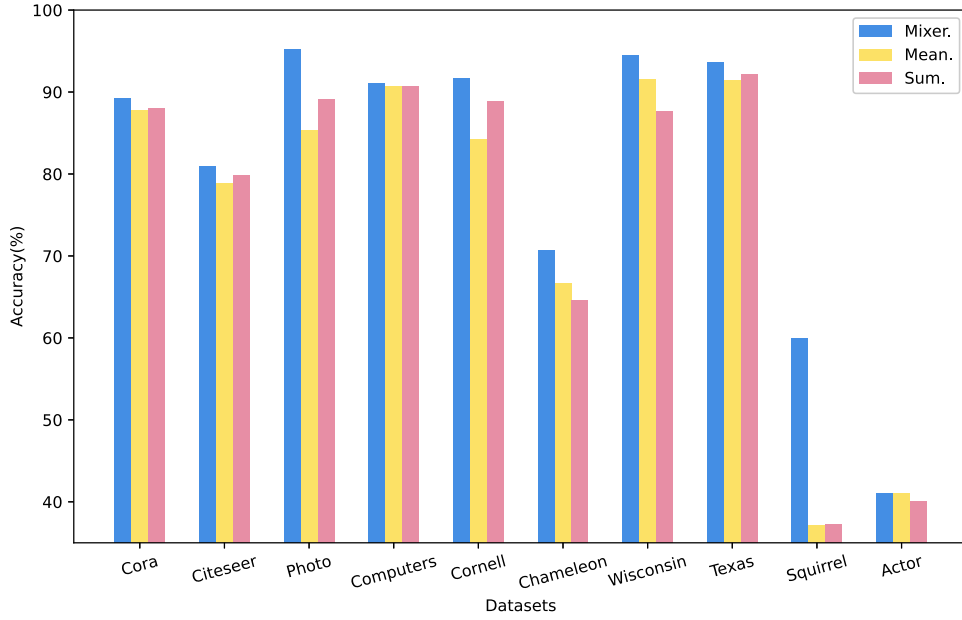


Fig. 6. The ablation study on the impact of adaptive mixing different simplices complexes.

Table 4

The statistics of graph classification datasets.

	PROTEINS	MUTAG	PTC-MR	IMDB-BIN	IMDB-MULTI
Graphs	1113	188	344	1000	1500
Nodes (max)	620	28	109	136	89
Nodes (avg.)	39.06	17.93	14.29	19.77	13.0
Edges (avg.)	72.82	19.79	14.69	13.06	65.93
Classes	2	2	2	2	3

formation on both homophily graphs and heterophily graphs. In addition, the improvement is more pronounced on heterophily graphs than on homophily graphs. This is probably caused by the easier fitting of the low-pass filters in homophily graphs and the Fourier expansion in our model allows for more complex filters. An interesting observation is that the improvement on larger graphs, e.g., Squirrel and Photo, is more than that on smaller graphs, e.g., Cornell and Texas. One possible reason is that the Fourier expansion shows stronger fitting capability as the interpolation points of the fitting function increase. These compelling results provide concrete evidence for the effectiveness of MFFN as a powerful learner for node-level predictions.

4.3. Graph classification

Datasets. We use five real-world graph datasets to evaluate our MFFN on the graph classification task, specifically as follows: PROTEINS, MUTAG, PTC-MR, and IMDB-Binary and IMDB-Multi. The statistics are summarized in Table 4.

Experimental Setting. We follow the same evaluation protocol of InfoGraph (Sun et al., 2020), conducting a 10-fold cross-validation procedure and reporting the maximum average validation accuracy across folds. We set the size of the hidden dimension of graph representations to 64. The following hyperparameters are adjusted for our proposed model: batch size $\in \{32, 64\}$, learning rate $\in \{0.0005, 0.001, 0.005, 0.01\}$ and dropout rate $\in \{0.0, 0.3\}$, respectively.

Baselines.

We compare MFFN with some baselines: (1) Graph representation learning methods including Graph kernel (GK) (Shervashidze et al.,

Table 5

Mean 10-fold cross-validation accuracy(%) and standard deviation(%) of MFFN on graph classification compared with baselines.

Method	PROTEINS	MUTAG	PTC-MR	IMDB-BIN	IMDB-MULTI
Graph-Level Methods					
GK	71.4 $\pm$ 0.3	81.7 $\pm$ 2.1	55.3 $\pm$ 1.4	65.9 $\pm$ 1.0	43.9 $\pm$ 0.4
WL	75.0 $\pm$ 3.1	80.7 $\pm$ 3.0	58.0 $\pm$ 0.5	72.3 $\pm$ 3.4	47.0 $\pm$ 0.5
DGCNN	61.3 $\pm$ 1.6	85.1 $\pm$ 1.7	58.6 $\pm$ 2.5	70.0 $\pm$ 10.9	47.8 $\pm$ 10.9
DCNN	75.5 $\pm$ 0.9	67.0 $\pm$ 1.5	56.6 $\pm$ 1.3	49.1 $\pm$ 1.4	33.5 $\pm$ 1.4
GIN	76.2 $\pm$ 2.8	89.4 $\pm$ 5.6	64.6 $\pm$ 7.0	75.1 $\pm$ 5.1	52.3 $\pm$ 2.8
CIN	77.0 $\pm$ 4.3	91.7 $\pm$ 6.1	67.2 $\pm$ 5.6	75.6 $\pm$ 3.7	52.7 $\pm$ 3.1
SIN	76.5 $\pm$ 3.4	91.2 $\pm$ 1.0	67.3 $\pm$ 2.6	75.6 $\pm$ 3.2	52.5 $\pm$ 3.0
Graph- and Node-level Methods					
GCN	75.2 $\pm$ 2.8	85.1 $\pm$ 5.8	63.1 $\pm$ 4.3	73.8 $\pm$ 3.4	51.9 $\pm$ 3.8
H2GCN	66.7 $\pm$ 6.9	86.1 $\pm$ 7.6	66.2 $\pm$ 5.9	77.0 $\pm$ 4.5	52.1 $\pm$ 3.7
APPNP	65.5 $\pm$ 10.3	89.8 $\pm$ 8.9	59.6 $\pm$ 6.5	75.2 $\pm$ 4.9	51.9 $\pm$ 2.8
HiGCN	77.0 $\pm$ 4.2	91.3 $\pm$ 6.4	66.2 $\pm$ 6.9	76.2 $\pm$ 5.1	52.7 $\pm$ 3.5
Transformer	66.3 $\pm$ 8.4	81.9 $\pm$ 9.7	57.3 $\pm$ 7.0	71.1 $\pm$ 3.8	45.8 $\pm$ 3.8
Graphormer	68.5 $\pm$ 2.3	82.5 $\pm$ 3.8	59.2 $\pm$ 4.6	73.5 $\pm$ 3.8	48.9 $\pm$ 2.3
NAGFormer	69.5 $\pm$ 3.4	81.4 $\pm$ 8.8	58.4 $\pm$ 5.6	72.1 $\pm$ 3.8	49.1 $\pm$ 1.8
MFFN	77.4 $\pm$ 1.9	92.5 $\pm$ 5.4	67.5 $\pm$ 2.4	77.3 $\pm$ 2.9	52.8 $\pm$ 2.4

2009), Weisfeiler-Lehman sub-tree kernel (WL) (Shervashidze et al., 2011), Diffusion convolutional neural networks (DCNN) (Atwood & Towsley, 2016), Deep Graph CNN (DGCNN) (Zhang et al., 2018), CIN (Bodnar et al., 2021a), and SIN (Bodnar et al., 2021b). (2) Graph and node representation learning methods including GCN, H2GCN, APPNP, HiGCN, Transformer, Graphormer, and NAGFormer.

Results. We show the results in Table 5 that MFFN achieves state-of-the-art results concerning baseline models on graph classification. Compared to graph kernel methods, MFFN surpasses GK by 9.8% on MUTAG and surpasses WL by 9.5% on PTC-MR. Furthermore, compared to some GNNs, MFFN improves by 3.5% over GCN and 5% over GraphSAGE on IMDB-BIN. Additionally, compared to graph Transformers, MFFN improves by 7.0% over Transformer and 3.9% over Graphormer on IMDB-MULTI. Finally, MFFN improves by 7.9% over NAGFormer and 0.4% over HiGCN on PROTEINS. These results provide concrete evidence for

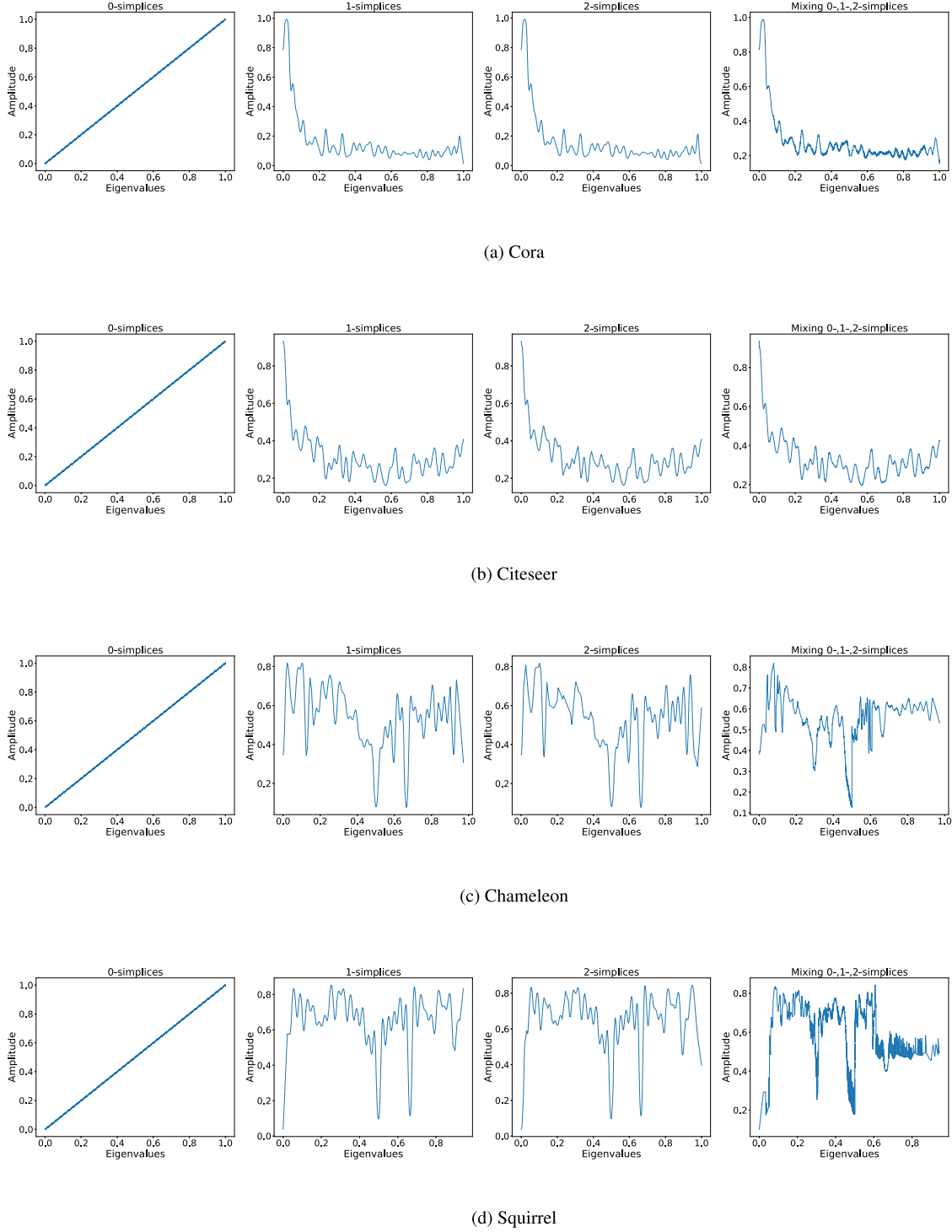


Fig. 7. The learned filters of MFNN on different datasets.

the effectiveness of MFNN as a powerful learner for graph classification, showing potential application prospects and good generalization capabilities.

4.4. Ablation study

To validate the effectiveness of our model for graph representation learning, we design three variant models for comparison in node classification:

1. w/o Fourier: not use Fourier expansions to transform the eigenvalues of Laplace matrices. Instead, we use the graph filter of GCN.
2. w/o Simplices: not use simplicial complexes to get different orders of Laplacians about the graph. Instead, we use the original Laplacian matrix L .
3. w/o Fourier and Simplices: not use simplicial complexes to get different orders of Laplacians about the graph and not use Fourier expansions to transform the eigenvalues of Laplace matrices. In this case, our model degenerates to GCN.

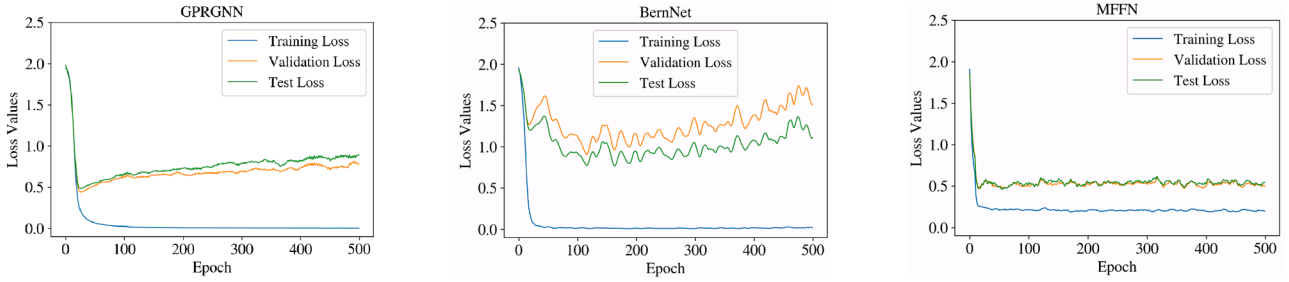


Fig. 8. MFFN's train/val/test loss with increasing epoch.

Table 6

Mean accuracy(%) $\pm$ 95% confidence interval of ablation study on the impact of cross-attention.

	w/o Fourier	w/o Simplicies	w/o Fourier and Simplicies	MFFN
Cora	87.00 $\pm$ 1.26	86.95 $\pm$ 0.89	86.39 $\pm$ 3.11	89.18$\pm$0.89
Citeseer	78.05 $\pm$ 0.99	77.00 $\pm$ 1.04	79.63 $\pm$ 0.71	80.90$\pm$0.76
Photo	93.60 $\pm$ 0.23	90.80 $\pm$ 0.86	93.78 $\pm$ 0.23	95.18$\pm$0.33
Computers	87.67 $\pm$ 0.89	90.84 $\pm$ 0.33	89.55 $\pm$ 0.55	91.05$\pm$0.26
Cornell	87.71 $\pm$ 2.46	87.21 $\pm$ 2.46	86.39 $\pm$ 3.11	91.64$\pm$2.62
Chameleon	50.72 $\pm$ 2.86	67.94 $\pm$ 1.42	56.85 $\pm$ 1.36	70.72$\pm$1.18
Wisconsin	92.13 $\pm$ 1.75	92.88 $\pm$ 2.25	94.40 $\pm$ 2.25	94.63$\pm$1.50
Texas	88.85 $\pm$ 2.30	82.79 $\pm$ 2.79	86.89 $\pm$ 2.30	93.61$\pm$2.13
Squirrel	39.61 $\pm$ 0.77	35.16 $\pm$ 4.03	32.58 $\pm$ 0.68	59.91$\pm$0.85
Actor	40.58 $\pm$ 0.77	39.91 $\pm$ 0.80	40.22 $\pm$ 0.64	41.07$\pm$1.17

The experimental results comparing MFFN with the above-mentioned variant models are presented in Table 6. Across all datasets, the lack of Fourier expansion leads to a slight decrease in performance, while the absence of complex features also results in a decline in model performance. Furthermore, the simultaneous absence of both Fourier expansion and complex features leads to a significant decrease in model performance, particularly on the Chameleon and Squirrel datasets. It is evident that Fourier and Simplex play a more crucial role in the Squirrel and Chameleon datasets compared to the Cora and Citeseer datasets. This can be attributed to the fact that the spectral filter of Squirrel and Chameleon is more challenging to learn.

We compare the proposed adaptive mixing 0-, 1-, 2-simplices with alternative aggregation functions, such as Mean. and Sum., as depicted in Fig. 6. We observe that mixing 0-, 1-, 2-simplices outperforms other functions on all datasets, indicating that aggregating information using attention adaptively is beneficial to learning more expressive node representations. Particularly, it outperforms other functions on Photo, Squirrel, and Chameleon, further confirming its positive role in learning more expressive node representations.

4.5. Visualization

Fig. 7 illustrates several filters learned by MFFN in the training process. On all datasets, 0-simplices show almost the same trend. Actually, 0-simplices in this work refer to self-rings on a graph. In this case, any adjacency matrix is an identity matrix, and all eigenvalues are the same. So the curve of all datasets shown in the first column is the same, which is essentially a linear function over the dot. On the contrary, 1-simplices and 2-simplices show the different structural properties of networks (e.g., homophily or heterophily). On Cora and Citeseer, MFFN learns two low-pass filters. On Chameleon and Squirrel, MFFN learns two comb-alike filters. At last, by combining different simplices, MFFN is capable of fitting arbitrary graph filters.

Fig. 8 shows the loss curves of our model, and two popular spectral methods including BernNet and GPRGNN on the Cora dataset. Clearly, we observe that BernNet's training loss first decreases and then reaches a relatively steady value. However, during this process, its validation and test losses exhibit a clear pattern of first decreasing and then in-

creasing, indicating the emergence of significant over-fitting as training progresses. In contrast, our method MFFN exhibits much more stable and consistent loss curves across the training, validation, and test sets. MFFN train multiple filters with guidance from diverse high-order patterns, while avoiding the possible overfitting induced by parameterizing a single filter specialized for a fixed first-order Laplacian matrix.

5. Conclusion

This paper proposes MFFN, which improves fitting capability and expressiveness power via Fourier expansion and simplicial complexes. Compared to previous methods, MFFN is capable of uncovering more intricate topological structures and approximating complex filters effectively. Experimental results from filter fitting, as well as node and graph classification tasks, demonstrate the advantages of MFFN. Looking ahead, a promising area for future research involves enhancing the efficiency of MFFN.

CRedit authorship contribution statement

Hui Yan: Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization; **Ling Guo:** Writing – original draft, Software, Methodology, Formal analysis; **Guoguo Ai:** Writing – review & editing, Validation, Methodology; **Xin Li:** Writing – review & editing, Supervision, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Abu-El-Hajja, S., Perozzi, B., Kapoor, A., Alipourfard, N., Lerman, K., Harutyunyan, H., Ver Steeg, G., & Galstyan, A. (2019). Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *International conference on machine learning (ICML)* (pp. 21–29).
- Akansa, S. (2023). Over-squashing in graph neural networks: A comprehensive survey. arXiv preprint arXiv:2308.15568.
- Atwood, J., & Towsley, D. (2016). Diffusion-convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 29.
- Balcilar, M., Renton, G., Héroux, P., Gatizère, B., Adam, S., & Honeine, P. (2021). Analyzing the expressive power of graph neural networks in a spectral perspective. In *International conference on learning representations (ICLR)*.
- Bastos, A., Nadgeri, A., Singh, K., Kanezashi, H., Suzumura, T., & Mulang, I. O. (2022). How expressive are transformers in spectral domain for graphs? In *Transactions on machine learning research*.
- Battiston, F., Cencetti, G., Iacopini, I., Latora, V., Lucas, M., Patania, A., Young, J.-G., & Petri, G. (2020). Networks beyond pairwise interactions: Structure and dynamics. *Physics Reports*, 874, 1–92.
- Bianchi, F. M., Grattarola, D., Livi, L., & Alippi, C. (2021). Graph neural networks with convolutional arma filters. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 44(7), 3496–3507.
- Bo, D., Shi, C., Wang, L., & Liao, R. (2023). Specformer: Spectral graph neural networks meet transformers. *International Conference on Learning Representations (ICLR)*.
- Bodnar, C., Frasca, F., Otter, N., Wang, Y., Lio, P., Montufar, G. F., & Bronstein, M. (2021a). Weisfeiler and leman go cellular: Cw networks. *Advances in Neural Information Processing Systems*, 34, 2625–2640.

- Bodnar, C., Frasca, F., Wang, Y., Otter, N., Montufar, G. F., Lio, P., & Bronstein, M. (2021b). Weisfeiler and lehman go topological: Message passing simplicial networks. In *International conference on machine learning* (pp. 1026–1037). PMLR.
- Cai, Y., Fang, G., & Li, P. (2021). A note on sparse generalized eigenvalue problem. *Advances in Neural Information Processing Systems (NeurIPS)*, 34, 23036–23048.
- Centola, D. (2010). The spread of behavior in an online social network experiment. *Science*, 329(5996), 1194–1197.
- Chen, J., Gao, K., Li, G., & He, K. (2023). Nagphormer: A tokenized graph transformer for node classification in large graphs. In *11th international conference on learning representations (ICLR)*.
- Chien, E., Peng, J., Li, P., & Milenkovic, O. (2021). Adaptive universal generalized pagerank graph neural network. In *9th international conference on learning representations (ICLR)*.
- Defferrard, M., Bresson, X., & Vandergheynst, P. (2016). Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in Neural Information Processing Systems (NeurIPS)*, 29.
- Diederik, P. K. (2015). Adam: A method for stochastic optimization. In *3rd international conference on learning representations (ICLR)*.
- Dwivedi, V. P., & Bresson, X. (2020). A generalization of transformer networks to graphs. In *CoRR*.
- Foupouagnigni, M. (2020). An introduction to orthogonal polynomials. In *Orthogonal polynomials: 2nd AIMS-volkswagen stiftung workshop* (pp. 3–24).
- Ganmor, E., Segev, R., & Schneidman, E. (2011). Sparse low-order interaction network underlies a highly correlated and learnable neural population code. *Proceedings of the National Academy of sciences*, 108(23), 9679–9684.
- Gasteiger, J., Bojchevski, A., & Günnemann, S. (2018). Predict then propagate: Graph neural networks meet personalized pagerank. In *International conference on learning representations (ICLR)*.
- Gehring, J., Auli, M., Grangier, D., Yarats, D., & Dauphin, Y. N. (2017). Convolutional sequence to sequence learning. In *International conference on machine learning* (pp. 1243–1252). PMLR.
- Grilli, J., Barabás, G., Michalska-Smith, M. J., & Allesina, S. (2017). Higher-order interactions stabilize dynamics in competitive network models. *Nature*, 548(7666), 210–213.
- He, M., Wei, Z., Xu, H. et al. (2021). Bernnet: Learning arbitrary graph spectral filters via bernstein approximation. *Advances in Neural Information Processing Systems (NeurIPS)*, 34, 14239–14251.
- Huang, Y., Zeng, Y., Wu, Q., & Lü, L. (2024). Higher-order graph convolutional network with flower-petals laplacians on simplicial complexes. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)* (pp. 12653–12661). (vol. 38).
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*.
- Li, Q., Han, Z., & Wu, X.-M. (2018a). Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*. (vol. 32).
- Li, Y., Yu, R., Shahabi, C., & Liu, Y. (2018b). Diffusion convolutional recurrent neural network: data-driven traffic forecasting. *ICLR*.
- Liu, H., & Wu, Z. (2010). Non-negative matrix factorization with constraints. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)* (pp. 506–511). (vol. 24).
- Niepert, M., Ahmed, M., & Kutzkov, K. (2016). Learning convolutional neural networks for graphs. In *International conference on machine learning* (pp. 2014–2023). PMLR.
- Pei, H., Wei, B., Chang, K. C., Lei, Y., & Yang, B. . Geom-GCN: Geometric graph convolutional networks. In *8th international conference on learning representations, ICLR 2020, addis ababa, ethiopia, april 26–30, 2020*.
- Qiu, J., Tang, J., Ma, H., Dong, Y., Wang, K., & Tang, J. (2018). Deepinf: Social influence prediction with deep learning. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery and data mining (KDD)* (pp. 2110–2119).
- Schaub, M. T., Benson, A. R., Horn, P., Lippner, G., & Jadbabaie, A. (2020). Random walks on simplicial complexes and the normalized hodge 1-laplacian. *SIAM Review*, 62(2), 353–391.
- Shervashidze, N., Schweitzer, P., Van Leeuwen, E. J., Mehlhorn, K., & Borgwardt, K. M. (2011). Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(9).
- Shervashidze, N., Vishwanathan, S., Petri, T., Mehlhorn, K., & Borgwardt, K. (2009). Efficient graphlet kernels for large graph comparison. In *Artificial intelligence and statistics* (pp. 488–495).
- Sun, F.-Y., Hoffman, J., Verma, V., & Tang, J. (2020). Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. In *8th international conference on learning representations (ICLR)*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations (ICLR)*.
- Wang, J., Zhang, B., Sang, Z., Liu, Y., Wu, S., & Miao, Q. (2018). Convergence of a modified gradient-based learning algorithm with penalty for single-hidden-layer feed-forward networks. *Neural Computing and Applications*, 32, 2445–2456.
- Wang, K., Zhang, G., Zhang, X., Fang, J., Wu, X., Li, G., Pan, S., Huang, W., & Liang, Y. (2024). The heterophilic snowflake hypothesis: Training and empowering gnn for heterophilic graphs. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 3164–3175).
- Wang, X., & Zhang, M. (2022). How powerful are spectral graph neural networks. In *International conference on machine learning (ICML)* (pp. 23341–23362).
- Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., & Weinberger, K. (2019). Simplifying graph convolutional networks. In *International conference on machine learning (ICML)* (pp. 6861–6871).
- Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K.-i., & Jegelka, S. (2018). Representation learning on graphs with jumping knowledge networks. In *International conference on machine learning (ICML)* (pp. 5453–5462).
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., & Liu, T.-Y. (2021). Do transformers really perform badly for graph representation? *Advances in Neural Information Processing Systems (NeurIPS)*, 34, 28877–28888.
- Zhang, M., & Chen, Y. (2018). Link prediction based on graph neural networks. *Advances in neural information processing systems*, 31.
- Zhang, M., Cui, Z., Neumann, M., & Chen, Y. (2018). An end-to-end deep learning architecture for graph classification. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*. (vol. 32).
- Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., & Koutra, D. (2020). Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 7793–7804.
- Zhu, M., Wang, X., Shi, C., Ji, H., & Cui, P. (2021). Interpreting and unifying graph neural networks with an optimization framework. In *Proceedings of the web conference (WWW)* (pp. 1215–1226).
- Zill, D. G. (2020). Advanced engineering mathematics. Jones & Bartlett Learning.